日本実習プロジェクト 第1フェーズ - AI履歴書スクリーニング Agent

1. 主なプロジェクトテーマ

プロジェクト名

採用担当者向け 履歴書一次選考・潜在人材発見支援 Agent

プロジェクトの位置づけ

本プロジェクトは、大量の履歴書を集中的に処理する必要がある採用担当者や企業人事を対象とし、履歴書

本プロジェクトはHRに代わって最終的な採用判断を行うことを目的としない。大量の反復的・機械的な一次
AIが補助することで、HRが少数の重要候補者の履歴書を丁寧に読み、人的判断により多くの時間を使えるよ

解決したい主な課題

採用担当者が短期間に数百件から千件規模の履歴書を受け取る場合、通常は短時間で目を通し、目立つ少数

1. HRが明らかに求人と合わない大量の履歴書を繰り返し読む必要があり、時間コストが高い。
2. 経歴が標準的ではないものの、関連能力や移転可能なスキルを持つ候補者が、キーワード不足、職種名

本プロジェクトの中心的な目標は、**人的判断を代替しない前提で、大量の履歴書に対して説明可能な一次ス
の選考時間を大幅に削減するとともに、少数の履歴書をより深く読めるようにし、潜在的に有望な人材の見**

---

2. 代替プロジェクトテーマ

プロジェクト名

求職者向け 求人マッチング・履歴書改善アシスタント

プロジェクトの位置づけ

ユーザーが自身の履歴書と希望求人のJDをアップロードすると、システムが両者のマッチ度を分析し、すて

代替案とした理由

このテーマは主テーマと技術基盤が近く、PDF解析、求人要件抽出、履歴書構造化、セマンティックマッチ

---

3. 対象ユーザー

主な対象ユーザー

- 企業の採用担当者
- 企業 HR
- 中小規模の採用チーム
- 短時間で大量の履歴書を処理する必要があるプロジェクト責任者または配属・利用部門

代表的な利用シーン

ある求人に短期間で 500〜1000 件の履歴書が届く。採用担当者はまず、明らかに基本条件を満たしていない
50〜100 件程度の「詳しく読む価値がある履歴書」を選ぶ必要がある。

システムが先に履歴書解析、求人マッチング、候補者の層別表示を行い、HR
は重要候補者に対して人的レビューを行う。

---

4. 対象ユーザーが現在抱えている具体的な課題

1. 履歴書数が多く、人的な一次選考に時間がかかる

採用担当者は学歴、経験年数、スキル、プロジェクト経験などを繰り返し確認する必要があり、多くの作業

2. 短時間の確認では重要情報を見落としやすい

HR が 1 件の履歴書に数十秒しか使えない場合、候補者のプロジェクト成果、学習力、異職種での経験、移転

3. 従来のキーワード検索は機械的すぎる

候補者が実際には関連経験を持っていても、履歴書内の表現が JD と異なることがある。

例：

- 求人側：「顧客要件分析」
- 候補者側：「顧客と業務フローを確認する業務を担当」

単純なキーワードマッチでは、両者の関連性を認識できない可能性がある。

4. HR によって一次選考基準が完全には一致しない

同じ求人でも、担当者の経験によって選考結果が異なる場合がある。システムが求人要件や評価根拠を明示

---

5. 既存製品・既存手法の不足点

1. 完全な人手による選考

個別事情を踏まえて判断できる一方、処理速度が遅く、大量の履歴書を長時間読み続けることで疲労や見落

2. 単純なキーワード検索

実装コストは低いが、類義表現、プロジェクト文脈、移転可能スキルを理解しにくく、表現は異なっていて

3. ハードルールだけに依存する自動選考

明らかに条件を満たさない履歴書を高速に除外できる一方、学歴、経験年数、職種名を絶対条件として扱い

---

6. このプロジェクトを選択した理由

1. 課題が現実的かつ明確

大量の履歴書一次選考は採用業務でよくある反復作業であり、対象ユーザー、入力、処理フロー、出力結果

2. AI を使う合理的な場面がある

求人要件や履歴書は自然言語で記述されるため、類義語、表現差、文脈差が存在する。AI
は単に「AI 機能を見せる」ためではなく、情報抽出、セマンティックマッチング、理由生成を補助する用途

3. プロジェクト価値を検証しやすい

初期検証では、以下の指標を利用できる。

- 1 バッチの履歴書処理時間が短縮されたか
- 履歴書の重要情報を正しく抽出できたか
- 高マッチ候補者の並び順が概ね妥当か
- システムが明確なマッチ根拠を提示できたか

---

7. プロジェクトにおける AI の具体的な役割

本プロジェクトでは、AI を使うこと自体を目的として不要な機能を追加しない。大規模言語モデルは主に、

1. 自然言語の求人票を解析する

求人票の記述形式は統一されていないため、大規模言語モデルを使って　JD
を統一された構造へ変換する。

2. 異なる形式の履歴書を理解する

大規模言語モデルを用いて、異なる形式の履歴書を統一された構造へ変換する。

3. セマンティックマッチングを行う

大規模言語モデルは、異なる表現の背後にある同一または類似の意味を理解できる。

例：

- JD：「API 認証」
  - 履歴書：「JWT を用いたログイン認証を実装」
- JD：「パフォーマンス最適化」
  - 履歴書：「キャッシュ導入により API 応答時間を 30% 短縮」
- JD：「バックエンド API 開発」
  - 履歴書：「注文サービス、パラメータ検証、DB 連携を担当」

単純な文字列マッチでは正確に対応関係を認識できない場合があるが、大規模言語モデルであれば文脈を踏

4. スキル移転と成長のエビデンスを発見する

大規模言語モデルを用いて、以下を分析する。

- 類似スキルを移転できる可能性があるか
- 新しい技術を学び、実際に成果へつなげた経験があるか
- 職務範囲が継続的に広がっているか
- 転職・転向や非標準的な経歴の中で実際の成果を出しているか

5. オープンな全体観察を行う

固定された評価次元だけでなく、大規模言語モデルが JD と履歴書全体を読み、あらかじめ設定したルールの
がさらに読む価値のある特徴」を探す。

---

8. プロジェクトの基本フロー

求人票を入力
↓
大規模言語モデルで JD を解析
↓
履歴書を一括アップロード
↓
履歴書テキストを抽出し、構造化解析
↓
求人マッチスコアを算出
↓
エビデンスポテンシャルスコアを算出

↓
オープンディスカバリースコアを算出
↓
3 つのスコアの理由とエビデンスを集約
↓
求人マッチスコア順に表示

---

9. 実装予定の主要機能

初版では、以下の 5 つの主要機能を実装する予定とする。

機能 1：求人票解析

HR が求人票（JD）を入力すると、システムが大規模言語モデルを使って職種名、主要スキル、実務経験、学

機能 2：履歴書一括解析

HR は複数の PDF 履歴書を一度にアップロードする。

システムは順次、以下を行う。

1. 履歴書から原文テキストを抽出する。
2. 大規模言語モデルを使って候補者の学歴、スキル、職歴、プロジェクト経験、実績を抽出する。
3. 異なる形式の履歴書を統一された構造化データへ整理する。

機能 3：候補者の 3 次元評価

システムは各候補者について、3 つの独立したスコアを出力する。

3.1 求人マッチスコア　求人マッチスコアは、候補者の現在の経歴と求人要件が直接どの程度一致しているか

主な評価内容：

- 主要スキルの一致
- 実務経験年数
- 学歴要件
- 関連実務経験
- プロジェクト経験
- 職務内容の意味的な一致

学歴、実務経験年数、明確なスキルなどは、プログラム側の固定ルールで算出する。

一方、「求人票と履歴書の表現は異なるが、実際の業務内容は近い」場合には、大規模言語モデルによる意

求人マッチスコアの主な目的は、HR が詳しく読む必要のある履歴書数を素早く減らすことである。

3.2 エビデンスポテンシャルスコア　エビデンスポテンシャルスコアは、候補者が求人要件を完全には満たし

主に以下を分析する。

- 類似スキルを移転できるか
- 新技術を学習し、実際にプロジェクトを完遂した経験があるか
- 職務責任が継続的に拡大しているか
- 転職・転向や非標準的経歴の中で実際の成果を出しているか
- 直接経験がなくても、それを代替できる関連経験があるか

各判断は必ず履歴書内の事実を引用し、モデルの「感覚」だけで採点してはならない。

このスコアは、固定ルールが機械的になりすぎることで、成長力やスキル移転力のある候補者を見落とすこ

3.3 オープンディスカバリースコア　オープンディスカバリースコアは、大規模言語モデルが求人票と履歴書
がさらに読む価値のある特徴」を発見できたかを表す。

大規模言語モデルが発見し得る例：

- 一見ばらばらに見える複数の経験が、実際には同じ中核能力を軸に積み上がっている
- 職種名は一般的でも、実際には複雑な責任を担っている
- 未経験タスクから独力でプロジェクトを完遂するまでの行動を複数回示している
- 標準的ではない経歴によって、特殊な分野横断スキルの組み合わせが形成されている

オープンディスカバリースコアはあくまで補助的な視点であり、候補者の真の将来性を確定するものではない

機能 4：評価理由と候補者レポート

各評価モジュールは同時に以下を出力する。

- スコア
- 短い評価理由
- 主な判断根拠
- 対応する履歴書原文のエビデンス

機能 5：一括結果表示

HR はフロントエンドで 1 件の JD を入力し、複数の履歴書を一括アップロードする。

システムは各履歴書について解析・評価を行い、求人マッチスコアの高い順に結果を表示する。

表示例：

候補者	求人マッチ	エビデンスポテンシャル	オープンディスカバリー
候補者 A	88	72	69
候補者 B	62	85	91
候補者 C	43	39	45

HR は任意の候補者を選択し、3 つのスコアの理由、エビデンス、総合レポートを確認できる。

求人マッチスコアによって読む履歴書数を絞り込みつつ、エビデンスポテンシャルスコアとオープンディスカ

---

10. 1〜1.5 か月以内の実施計画

第 1 週：データ構造と求人解析を確定

- デモ用の具体的な職種を 1 つ決める
- 求人解析後に保存すべき項目を明確にする
- 履歴書解析後に保存すべき項目を明確にする
- 求人票解析機能を完成する
- 3 件の異なる JD で解析結果をテストする

週次完了条件：

JD を入力すると、統一された構造化求人データを得られる。

第 2 週：履歴書解析を完成

- PDF 履歴書のテキスト抽出を実装
- 大規模言語モデルで履歴書の重要情報を抽出
- 学歴、スキル、職歴、プロジェクト経験、実績を処理
- 5〜10 件の異なる形式の模擬履歴書でテスト
- 大規模言語モデルによる情報欠落や捏造がないか確認

週次完了条件：

1 件の履歴書をアップロードすると、統一された構造化候補者データを得られる。

第 3 週：求人マッチスコアを完成

- 学歴、経験年数、明確なスキルの固定ルール評価を実装
- 求人職務と候補者経験のセマンティックマッチングを実装
- ルール評価と意味評価を統合し、求人マッチスコアを生成
- 求人マッチスコア、評価理由、履歴書エビデンスを出力
- マッチ度の異なる模擬履歴書を使い、スコア変化をテスト

週次完了条件：

構造化求人データと構造化履歴書データから求人マッチスコアを生成できる。

第 4 週：2 種類のポテンシャル評価を完成

- エビデンスポテンシャルスコアを実装
- スキル移転、学習成果、成長経験、非標準経歴での成果を分析
- オープンディスカバリースコアを実装
- 大規模言語モデルに JD と履歴書全体を読ませ、事前ルール外の特徴を探索
- 2 種類の評価とも理由と原文エビデンスを出力

- 同一履歴書を複数回テストし、スコアの安定性を確認

週次完了条件：

各候補者に求人マッチ、エビデンスポテンシャル、オープンディスカバリーの 3 スコアを付与できる。

第 5 週：バッチ処理とフロントエンド表示を完成

- 単一履歴書分析をバッチループ処理へ変更
- 1 回 10〜30 件の履歴書アップロードに対応
- 求人マッチスコア順に並べる
- Gradio で簡易画面を作成
- 候補者名と 3 スコアを表示
- 候補者を選択すると、評価理由、エビデンス、総合レポートを表示

週次完了条件：

HR が 1 件の JD を入力し、履歴書を一括アップロードして、バッチ評価結果を確認できる。

第 6 週：テストと調整

- 10〜30 件の模擬または匿名化履歴書で全体テスト
- 求人解析・履歴書解析の正確性を確認
- 求人マッチスコアが人の想定と概ね一致するか確認
- エビデンスポテンシャルスコアが実際にスキル移転や成長の根拠を見つけられるか確認
- オープンディスカバリーで根拠のない過度な称賛が発生していないか確認
- プロンプトと評価ルールを調整
- 各大規模言語モデル呼び出しの Token 消費を記録
- テスト記録、デモ資料、プロジェクト説明を完成

時間に余裕があれば、複数の大規模言語モデル評価呼び出しを統合し、重複 Token 消費を削減する。

この項目は後続最適化とし、MVP の基本完成には影響しない。

---

11. 初版 MVP（Minimum Viable Product）の範囲

MVP で必須とする内容

1. 1 件の求人票を入力できる。
2. 大規模言語モデルで求人票を構造化データへ解析できる。
3. 1 回 10〜30 件のテキスト型 PDF 履歴書をアップロードできる。
4. 履歴書テキストを抽出し、構造化候補者データへ解析できる。
5. 各候補者に求人マッチスコアを出力できる。
6. 各候補者にエビデンスポテンシャルスコアを出力できる。
7. 各候補者にオープンディスカバリースコアを出力できる。

8. 各スコアとともに評価理由と履歴書エビデンスを出力できる。
9. 候補者を求人マッチスコアの高い順に表示できる。
10. 単一候補者について 3 つの評価理由と総合レポートを確認できる。
11. バッチ処理中に解析失敗した履歴書についてエラーを表示し、他の履歴書処理は継続できる。

---

12. MVP 稼働後の拡張機能

初版の基本フローが安定稼働した後、時間に応じて以下を追加する。

1. JD 解析結果の手動調整

JD 解析後、HR がシステム抽出結果を確認し、モデルの解析誤りを修正できるようにする。

HR が可能な操作：

- システムが抽出した求人要件を修正
- モデルが見落とした評価項目を追加
- 現在の求人に不適切な評価項目を削除
- 各評価項目の重みを設定または調整

この機能はモデル解析結果と重みを修正するためのものであり、MVP
の必須機能とはしない。

2. 重点レビューの自動マーキング

HR が重点レビュー用のスコア閾値を設定できる。

システム評価後、求人マッチスコアが閾値以上の候補者を自動的に以下としてマークする。

重点レビュー

このマークは HR が読む順番を決めるためだけに使用し、候補者が一次選考を通過したことを意味しない。

3. 「一次選考通過」の手動チェック

候補者一覧および候補者詳細に「一次選考通過」のチェック項目を追加する。

HR が候補者情報を確認後、手動で通過をチェックできる。チェック後の状態：

一次選考通過済み

これは HR の人的レビュー結果を記録するだけであり、AI が自動決定するものではない。

4. その他の後続拡張

- ローカルデータベース保存
- 候補者詳細と原本履歴書の連動表示
- 履歴タスク表示

- モデル呼び出し最適化

---

13. プロジェクトの特徴

本プロジェクトの主な特徴は、単なる履歴書マッチングだけではなく、各候補者に対して相互に独立した3つの評価軸を提供する点にある。

1. **求人マッチスコア**：候補者の現在の経歴と求人要件が直接どの程度一致するかを判断する。
2. **エビデンスポテンシャルスコア**：求人上の不足を補う可能性のある、スキル移転、学習力、成長のエビ
3. **オープンディスカバリースコア**：固定ルール外にある、HRがさらに読む価値のある特殊な経験や能力

この3つの評価軸は、それぞれ「現在どの程度マッチしているか」「不足を補える潜在的エビデンスがあるか
の一次選考と候補者レビューを共同で支援する。